2026-9-27 Linux网络--概念中

- 1. 从局域网通信到跨网络通信
  - 1.1 跨网络通信新增的问题
  - 1.2 回顾协议栈
- 2. IP 地址与 MAC 地址
  - 2.1 IP 地址标识网络层终点
    - 2.1.1 IPv4 的表示方式
    - 2.1.2 IP 是逻辑地址
  - 2.2 MAC 地址标识链路邻居
  - 2.3 IP 与 MAC 的分工
    - 2.3.1 为什么不能只保留一个地址
  - 2.4 从命令输出观察两类地址
    - 2.4.1 Linux 中的 ifconfig
    - 2.4.2 Windows 中的 ipconfig /all
- 3. 路由器如何完成跨网络转发
  - 3.1 先判断目标是否在本地网络
  - 3.2 主机只决定第一跳
  - 3.3 路由器如何重新封装数据
    - 3.3.1 拆掉旧链路帧
    - 3.3.2 保留端到端信息，调整下一跳信息
    - 3.3.3 每一跳都换 MAC 的原因
- 4. IP 为什么能屏蔽底层网络差异
  - 4.1 不同链路的实现并不统一
  - 4.2 IP 提供统一的虚拟互联网层
- 5. 从主机定位到进程定位
  - 5.1 数据到达主机还不是最终目的
  - 5.2 端口是传输层的进程定位信息
    - 5.2.1 端口范围
    - 5.2.2 端口为什么不直接使用 PID
  - 5.3 源端口与目的端口
- 6. TCP、UDP 与 Socket
  - 6.1 TCP 与 UDP 的直观区别
  - 6.2 Socket 是网络通信端点
  - 6.3 四元组标识一次双向通信
- 7. 一次跨网络通信的完整主线
  - 7.1 发送端如何封装

    - 7.2 路由器如何处理
    - 7.3 接收端如何解包
- 8. 小结与边界
    - 8.1 核心结论
    - 8.2 与后续内容的衔接

1. 从局域网通信到跨网络通信

上一部分解决了数据如何在同一局域网内逐层封装，并通过 MAC 地址交给目标网卡的问题。但如果目标主机不在当前局域网，仅靠 MAC 地址就无法完成通信。

本部分沿着参考图的主线，回答三个连续的问题：

1. 跨网络通信为什么需要 IP 地址？
2. 路由器如何把数据从源网络逐跳转发到目标网络？
3. 数据到达主机后，如何继续交给正确的进程？

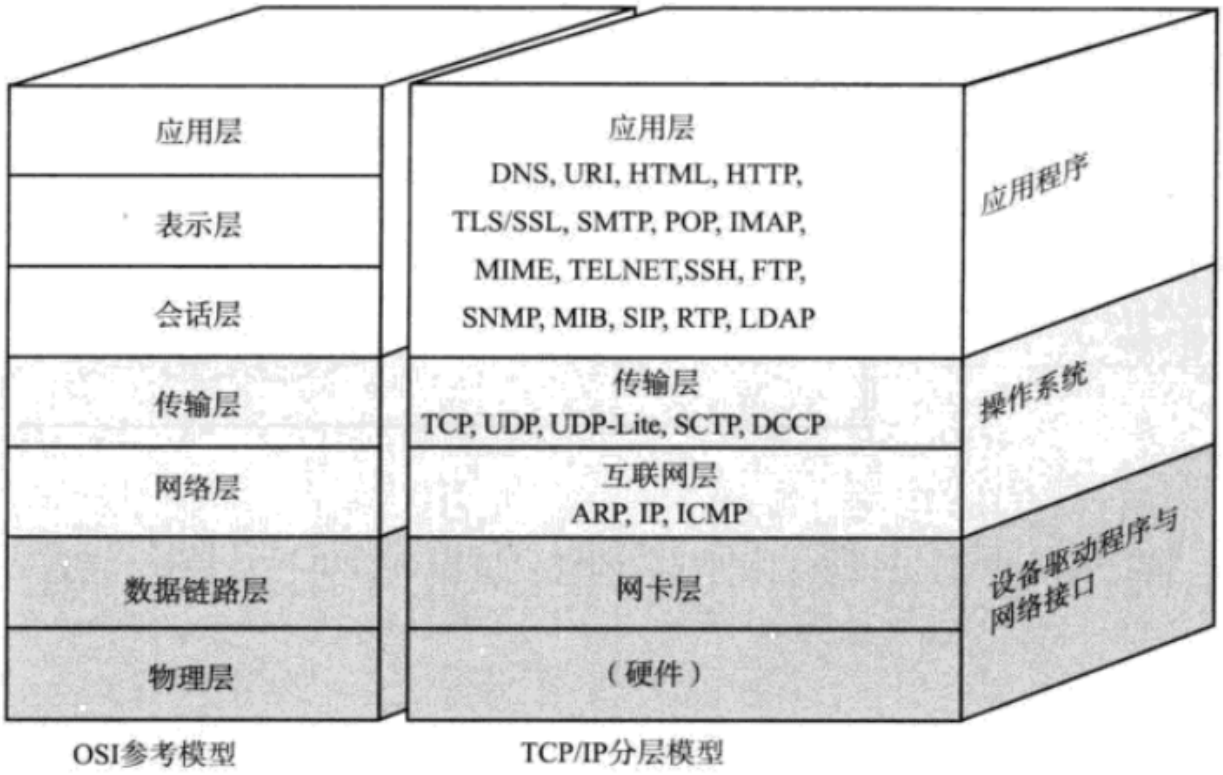
1.1 跨网络通信新增的问题

一台主机只知道自己在哪个局域网中，并不天然知道整个互联网的完整拓扑。发送数据前，网络层至少要判断：

- 目标主机是否位于当前局域网；
- 如果目标不在当前网络，应该把数据先交给哪个路由器；
- 路由器收到数据后，下一步应该转发到哪里；
- 不同链路技术之间如何互相兼容；
- 数据到达目标主机后，应该交给哪个上层协议和进程。

局域网通信解决的是“相邻链路之间如何交付”，跨网络通信还需要解决“最终目标在哪里”和“下一跳往哪里走”。

1.2 回顾协议栈



跨网络通信仍然沿协议栈进行：

```
应用层：产生业务数据
传输层：加入端口和传输协议报头
网络层：加入源 IP、目的 IP 等网络层报头
数据链路层：加入当前链路的 MAC 报头和帧尾
物理层：转换为比特流并发送
```

其中，网络层报头在整个通信过程中大体保持“源到目的”的语义，而数据链路层报头只负责当前这一段链路，因此会随着经过的路由器不断更换。

2. IP 地址与 MAC 地址

2.1 IP 地址标识网络层终点

IP 地址用于在网络层标识通信端点，并支持网络间的路径选择。当前默认讨论的 IP 通常是 IPv4。

2.1.1 IPv4 的表示方式

IPv4 地址长度为 32 位，即 4 字节，通常写成点分十进制形式：

```
192.168.2.2
172.168.2.2
```

每个十进制数字对应 1 字节，取值范围是 `0-255`。IPv6 使用更长的地址空间，但基本职责仍然是标识网络层端点。

2.1.2 IP 是逻辑地址

IP 地址通常由网络配置或 DHCP 分配，不跟随网卡硬件固定。更换网络环境时，主机可能需要获得新的 IP 地址。

IP 不描述“数据下一段具体从哪块网卡出去”，而是描述：

- 数据从哪里开始；
- 最终要到哪里；
- 网络层应该选择哪条路径接近目标。

因此，IP 地址更适合理解为**最终目标的逻辑定位**。

2.2 MAC 地址标识链路邻居

MAC 地址属于数据链路层，用于在同一链路或同一局域网内标识网卡。经典以太网帧中会携带源 MAC 和目的 MAC：

```
源 MAC | 目的 MAC | 网络层数据 | 帧尾
```

MAC 地址只解决“这一帧在当前链路上应该交给哪个相邻设备”，不能直接描述跨越多台路由器的最终路径。

2.3 IP 与 MAC 的分工

对比项	IP 地址	MAC 地址
所属层次	网络层	数据链路层
地址性质	逻辑地址，可配置	网卡地址，通常出厂分配
主要范围	跨网络的端到端定位	当前链路的下一跳定位

对比项	IP 地址	MAC 地址
跨路由器时	与 MAC 相比基本保持源、目的语义	每一跳都可能更换
主要作用	选择最终目标和路径	完成当前链路的帧交付

参考图使用“旅行”来类比：

- IP 相当于“从东土大唐去往西天”的最终目的地；
- MAC 相当于“从车迟国去往女儿国”的下一站；
- 路由器每到一个新网络，都要重新确定下一站。

可以用一句话概括：

> IP 管最终目标，MAC 管下一跳。

2.3.1 为什么不能只保留一个地址

如果只用 MAC：

- 每台路由器都要知道全世界所有网卡的地址；
- 路由表会失去层次，难以聚合和维护；
- 不同局域网、无线网络、令牌环等链路技术之间缺少统一抽象。

如果只用 IP：

- 网络层知道最终目标，却无法直接把 IP 数据报交给物理介质；
- 每个链路仍需把“下一跳 IP”转换为“下一跳链路地址”。

因此需要两套地址共同工作：IP 提供远程目标，MAC 完成本地交付。

2.4 从命令输出观察两类地址

2.4.1 Linux 中的 `ifconfig`

参考图同时给出了 Linux 的 `ifconfig` 输出。可以先关注两类字段：

```
inet 172.28.71.17  netmask 255.255.240.0  broadcast 172.28.79.255
ether 00:16:3e:10:cc:36  txqueuelen 1000  (Ethernet)
```

- `inet` 是接口的 IPv4 地址，属于网络层；
- `netmask` 用来辅助判断目标是否在当前网段；
- `broadcast` 是当前网段的广播地址；
- `ether` 是接口的 MAC 地址，属于数据链路层。

现代 Linux 也可以使用 `ip addr` 和 `ip link show` 查看同类信息。命令形式不同，但 IP 与 MAC 的层次分工不变。

2.4.2 Windows 中的 `ipconfig /all`

Windows 的 `ipconfig /all` 会把同类信息集中列出：

- IPv4 地址；
- 子网掩码；
- 默认网关；
- 物理地址（MAC）；
- DNS 服务器。

其中，IPv4 地址和默认网关用于网络层判断及跨网段转发，物理地址用于当前链路的帧交付。DNS 服务器则帮助应用把域名解析为 IP 地址，属于后续应用层内容。

3. 路由器如何完成跨网络转发

3.1 先判断目标是否在本地网络

主机会结合自己的 IP 地址和子网掩码判断目标 IP 是否属于当前网络。

- 目标在同一网段：主机可以直接通过本地链路向目标发送；
- 目标不在同一网段：主机把数据交给默认网关或匹配到的路由条目。

例如，主机 A 位于 `192.168.2.0/24`，目标 B 的地址是 `172.168.2.2`。因为目标不在 A 的本地网段，A 不能直接向 B 发送以太网帧，而要先把数据交给本地路由器。

3.2 主机只决定第一跳

发送方通常不需要预先知道完整路径。它只需要知道：

- 最终目标 IP；
- 当前应交给哪个下一跳路由器。

网络层根据路由表决定下一跳，数据链路层再为这一跳构造新的帧。下一台路由器继续作出自己的下一跳决定。

在以太网中，路由器还需要知道下一跳 IP 对应的 MAC 地址。IPv4 网络通常通过 ARP 完成“下一跳 IP 到链路地址”的解析：

- 目标在同一网段时，解析目标主机的 IP；
- 目标在其他网段时，解析本地网关的 IP。

可以概括为：

```
主机 A -> 当前 LAN 的网关
       -> 路由器的下一跳
       -> ...
       -> 目标 LAN 的路由器
       -> 主机 B
```

每一台路由器只需要知道“下一步往哪里走”，不必保存整个世界所有主机的完整路径。

3.3 路由器如何重新封装数据

3.3.1 拆掉旧链路帧

路由器收到帧后，会处理数据链路层报头，取出其中的 IP 数据报。链路层只对当前链路负责，所以原来的 MAC 帧不能直接用于下一条链路。

3.3.2 保留端到端信息，调整下一跳信息

路由器查看 IP 报头中的目的 IP，并查询路由表。确定下一跳后，它会为新的链路重新封装：

```
原有：MAC_A -> MAC_router1 | IP_A -> IP_B | 传输层数据
下一跳：MAC_router1 -> MAC_router2 | IP_A -> IP_B | 传输层数据
```

对比可知：

- 目的 IP 仍指向主机 B，网络层继续把它作为最终目标；
- 源、目的 MAC 会换成当前链路上的设备地址；
- 下一台路由器重复“解包到网络层，再重新封装链路帧”的过程。

严格来说，IP 报头也可能发生变化，例如 TTL 递减、分片或 NAT 改写。因此，在普通的未改写路由过程中，可以近似说“IP 基本不变”，但不能把它理解成任何字段都绝不改变。

3.3.3 每一跳都换 MAC 的原因

MAC 地址描述的是当前链路两端的网卡，不具有跨链路保持不变的职责。只要数据进入了新链路，就需要使用新链路上的源、目的 MAC。

因此：

- IP 地址表达“最终从哪里到哪里”；
- MAC 地址表达“这一帧从一个相邻设备到哪里”；
- 路由器连接多段链路，所以每一跳都要重新封装。

4. IP 为什么能屏蔽底层网络差异

4.1 不同链路的实现并不统一

真实网络中存在以太网、令牌环、无线局域网等多种链路技术。它们的帧格式、介质和介质访问方法不同，不能直接互操作。

如果应用和数据必须直接适配每种链路：

- 应用迁移到另一种网络会非常困难；
- 每增加一种链路技术，上层协议就要修改；
- 不同厂商、不同网络之间难以形成统一互联网。

4.2 IP 提供统一的虚拟互联网层

IP 在网络层提供统一的地址和转发抽象。上层不需要关心当前经过的是以太网、无线网络还是点对点链路，只需把 IP 数据报交给网络层。

路由器负责在不同链路之间进行转换：

```
上层看到的：统一的 IP 网络
底层实际使用的：以太网、无线链路、专线等
```

这就是“IP over everything”的思想：不同底层链路可以承载同一种 IP 数据报，因此 IP 能够把许多异构局域网连接成一个逻辑上的统一网络。

5. 从主机定位到进程定位

5.1 数据到达主机还不是最终目的

IP 负责把数据送到目标主机，但用户真正使用数据的是浏览器、聊天软件、下载工具等进程。

因此需要继续回答：

> 同一台主机上运行着许多进程，这份数据到底应该交给谁？

网络层通过目的 IP 找到主机，传输层再通过端口把数据交给对应进程。

5.2 端口是传输层的进程定位信息

端口号是传输层协议的重要内容。TCP 和 UDP 报头都包含源端口和目的端口。

5.2.1 端口范围

端口号是 16 位整数，取值范围为 `0-65535`：

- `0-1023`：常见知名端口或系统端口区域，例如 HTTP 常使用 80，HTTPS 常使用 443，SSH 常使用 22；
- `1024-65535`：常见动态端口区域，客户端进程使用的临时端口通常由操作系统分配。

具体系统对端口范围和临时端口区间的配置可能不同，不能把所有 `1024` 以上的端口都理解为只供客户端使用。

5.2.2 端口为什么不直接使用 PID

PID 是操作系统用来管理进程的标识，但它不适合作为网络协议中的固定服务标识：

- 进程重启后 PID 可能变化；
- 并非所有进程都需要参与网络通信；
- 网络协议需要跨主机、跨操作系统表达“目标服务”；
- 把网络协议直接绑定到系统 PID，会让网络设计和进程管理过度耦合。

端口描述的是“目标进程在网络中的服务入口”，PID 描述的是“操作系统当前管理的是哪个进程”。两者解决的是不同层面的问题。

一般情况下，一个端口在同一时间和同一传输协议下由一个进程绑定；一个进程也可以绑定多个端口。因此，端口和进程不是固定的一对一 PID 映射。

5.3 源端口与目的端口

传输层报头中通常同时存在两个端口：

- 源端口：发送方进程的端口；
- 目的端口：目标主机上接收方进程的端口。

例如，浏览器访问 HTTPS 服务时，客户端可能使用一个临时源端口，而目的端口通常是 443。

完整的分用过程可以理解为：

```
目的 MAC -> 找到当前链路的接收设备
目的 IP  -> 找到网络中的目标主机
目的端口 -> 找到目标主机中的目标进程
```

6. TCP、UDP 与 Socket

6.1 TCP 与 UDP 的直观区别

参考图的传输层位置同时标出了 TCP 和 UDP。

特性	TCP	UDP
连接	面向连接	无连接

特性	TCP	UDP
可靠性	提供可靠传输机制	不保证可靠到达
数据形式	面向字节流	面向数据报
典型场景	文件传输、网页连接等	实时音视频、简单查询等

TCP 与 UDP 的具体机制会在后续协议专题中展开，这里只需先建立“二者都使用端口定位进程，但传输语义不同”的认识。

6.2 Socket 是网络通信端点

只使用 IP，可以找到一台主机；只使用端口，还不能说明是哪台主机上的哪个进程。

把两者组合起来：

```
IP 地址   -> 定位网络中的主机
端口号    -> 定位该主机上的网络进程
IP + 端口 -> 定位一个网络通信端点
```

通常把 `IP + Port` 所表示的通信端点称为 Socket。

需要注意，Socket 既可以是操作系统提供的编程接口，也可以指一次通信中的端点抽象。本部分重点理解“端点”概念，具体的 `socket()`、`bind()`、`listen()` 等 API 属于后续编程内容。

6.3 四元组标识一次双向通信

一次完整的网络连接通常需要同时知道双方的 IP 和端口：

```
{srcIP, srcPort, dstIP, dstPort}
```

例如：

```
{192.168.2.3, 8888, 192.168.2.4, 9999}
```

四元组能够区分同一对主机上的不同连接。即使多个客户端访问同一个服务器端口，也可以通过客户端 IP、客户端端口等差异形成不同的四元组。

因此可以这样分工：

- IP 解决“到哪台主机”；
- 端口解决“交给哪个进程”；
- 四元组解决“是哪一次通信”。

7. 一次跨网络通信的完整主线

7.1 发送端如何封装

以主机 A 向主机 B 发送数据为例：

1. 应用层产生业务数据；
2. 传输层加入源端口、目的端口以及 TCP 或 UDP 报头；
3. 网络层加入源 IP、目的 IP 等报头字段；
4. 数据链路层加入当前链路的源 MAC、目的 MAC；
5. 网卡把帧转换为比特流并发送。

如果目的主机不在本地网段，目的 MAC 通常指向网关路由器，而目的 IP 仍然指向最终主机 B。

7.2 路由器如何处理

路由器每经过一跳，都执行类似过程：

1. 接收当前链路的帧；
2. 去掉链路层报头，读取 IP 数据报；
3. 查看目的 IP，查询路由表；
4. 确定下一跳；
5. 重新加上下一跳链路的报头；
6. 从新的接口发送。

数据在路由器内部主要上升到网络层，因此中间路由器通常不需要理解应用层业务数据。

7.3 接收端如何解包

目标主机收到帧后，按协议栈自下而上处理：

1. 数据链路层检查目的 MAC，接收当前帧；
2. 网络层检查目的 IP，并把有效载荷交给 TCP 或 UDP；
3. 传输层根据目的端口，把数据交给目标进程；
4. 应用层取得业务数据并完成处理。

完整过程可以简写为：

```
应用数据
  -> 加传输层报头
  -> 加网络层 IP 报头
  -> 加当前链路 MAC 报头
  -> 路由器逐跳拆帧、查路由、重新封装
  -> 目标主机逐层去报头
  -> 根据目的端口交给进程
```

8. 小结与边界

8.1 核心结论

跨网络通信可以归纳为以下关系：

1. 数据链路层用 MAC 地址完成当前链路的帧交付；
2. 网络层用 IP 地址表达最终位置，并通过路由选择下一跳；
3. 路由器每经过一跳都要拆掉旧链路帧，再构造新链路帧；
4. IP 屏蔽不同底层链路的差异，形成统一的虚拟互联网络；
5. 数据到达主机后，传输层通过端口继续定位进程；
6. `IP + Port` 构成 Socket 端点，四元组标识一次双向通信。

8.2 与后续内容的衔接

参考图在解释端口和四元组后，把问题继续引向“操作系统如何把数据交给指定端口对应的进程”。

后续 Socket 编程将进一步处理：

- 网络字节序；
- `socket()`、`bind()`、`listen()`、`accept()`、`connect()`；
- `sockaddr` 与 `sockaddr_in`；
- 应用进程如何创建、绑定并管理网络端点。

因此，本笔记负责建立跨网络通信和进程定位的宏观模型，不提前展开具体 API 的调用细节。